llama.cpp / GGUF 前置知识

> 这份文档面向对"本地跑大模型"有好奇但没有技术背景的普通读者。

> 🎯 **读完这篇你能**：理解为什么要在自己电脑上跑大模型、llama.cpp 和 Ollama 是什么关系、GGUF 量化格式解决什么问题、k-quant 后缀（Q4_K_M）代表什么含义。

一、为什么要在自己的电脑上跑大模型?

用 ChatGPT 或 Claude，你的每一句话都要发到云端服务器。好处是省事，代价是：要联网、要付费、数据不在自己手里。

本地推理就是反过来的——模型文件下载到你的电脑上，所有计算都在本地完成。代价是你的电脑得有足够的算力和内存。好处：断网也能用、不花钱、数据不出门。

直到 2023 年初，这还只是极客的玩具。一个保加利亚程序员用纯 C++ 写了 llama.cpp，从此改变了这一切。

二、GGUF 量化：灵魂是"压缩"

大模型很大。LLaMA-7B 的原始权重文件约 13GB。这对消费级电脑已经是极限了——普通人没有 A100。

量化就是"存少几个比特"。原始模型每个权重存成 16 位浮点数（FP16），量化后每个权重只用 4 位、6 位、或者 8 位。类似把照片从 BMP 存成 JPEG——肉眼看不出差别，文件小了好几倍。

GGUF（GGML Unified Format）是 llama.cpp 生态使用的模型文件格式。一个 GGUF 文件包含：

1. **模型权重**——经过量化的神经网络参数
2. **元数据**——tokenizer 类型、模型架构、上下文长度等，全部存在文件头部

GGUF 取代了早期的 GGML 格式（老格式没有元数据，全靠文件名猜，经常出兼容性问题）。到 2026 年，GGUF 已成为本地 LLM 分发的行业标准。

三、k-quant：普通程序员"烹饪"出来的量化方案

Iwan Kawrakow，一个医学物理背景的程序员，在 2023 年 6 月向 llama.cpp 提交了一套新的量化方案，叫 **k-quants**（以他的名字 K 命名）。

他本人在 GitHub 上形容自己做量化是"cooked up"（随手鼓捣）。没有任何学术论文——纯靠数学直觉和反复调参。

k-quant 系列的含义：

后缀	每权重比特数	大约文件大小（7B 模型）	质量
Q2_K	~2.6 bit	2.5 GB	极限压缩，能跑就行
Q3_K	~3.4 bit	3.2 GB	勉强可用
Q4_K_M	~4.5 bit	4.4 GB	**最推荐的性价比之选**
Q5_K_M	~5.5 bit	5.2 GB	质量很好
Q6_K	~6.6 bit	6.3 GB	接近无损
Q8_0	8 bit	7.7 GB	近乎原始

`_M` 和 `_S` 后缀代表 Medium 和 Small——同样比特数下，M 版本给关键层分配更多精度，体积稍大但质量更好。

四、llama.cpp 是什么、不是什么

是：

- 一个用纯 C/C++ 写的 LLM 推理引擎，没有 Python 依赖，没有 PyTorch
- 一个命令行程序，你给它 GGUF 文件，它输出文本
- 整个本地推理生态的"地基"——Ollama、LM Studio、Jan、GPT4All 的底层引擎全是它

不是：

- 不是一个模型——它不训模型，只跑模型
- 不是一个 GUI 应用——它只有命令行，没有窗口
- 不是一个服务——它的 server 模式是后来加的

五、一人生态，三层架构

llama.cpp 的生态像一棵树：

```
Georgi Gerganov（创始人）
  │
  ├── llama.cpp（核心引擎，纯 C++）
```

```
│      │
│      ├── GGUF 格式（模型打包标准）
│      ├── k-quant 量化方案
│      └── 多硬件支持（CPU/CUDA/Metal/Vulkan/ROCm）
│
├── 上层产品（基于 llama.cpp）
│      ├── Ollama（命令行 + API，最流行）
│      ├── LM Studio（GUI，体验最好）
│      ├── Jan（开源 GUI）
│      ├── GPT4All（消费级优化）
│      └── text-generation-webui（功能最全的 Web UI）
│
└── 生态工具
       ├── TheBloke（模型分发，HuggingFace 上量最多的 GGUF 模型）
       ├── llama-cpp-python（Python 绑定）
       └── candle（Rust 实现，也能加载 GGUF）
```

六、关键时间线

- **2022.09** — whisper.cpp 出现，GGML 张量库诞生
- **2023.03** — LLaMA 模型泄露，llama.cpp 第一版发布（"在 MacBook 上跑 LLaMA"）
- **2023.06** — k-quant 方案出现
- **2023.08** — GGUF 替代 GGML，成为标准格式
- **2024** — FlashAttention、投机解码、全 GPU 后端矩阵完善
- **2025** — 多模态支持（图片理解）、DeepSeek MLA 支持
- **2026.02** — ggml.ai 被 Hugging Face 收购，llama.cpp 保持 MIT 开源

七、我应该用哪个？

如果你是……	用……	为什么
开发者，想精确控制参数	llama.cpp 原版	最快、最多控制
想一行命令跑模型	Ollama	最省事
不想碰命令行	LM Studio	GUI 最好
注重隐私，全开源	Jan	离线优先
有 NVIDIA 显卡，追求极致速度	ExLlamaV3	CUDA 专属优化

只想写 Python	llama-cpp-python	API 最简

参考来源

- llama.cpp GitHub — https://github.com/ggml-org/llama.cpp
- GGUF 格式规范 — https://github.com/ggml-org/ggml/blob/master/docs/gguf.md
- k-quants PR #1684 — https://github.com/ggml-org/llama.cpp/pull/1684
- ggml.ai joins Hugging Face — https://huggingface.co/blog/ggml-joins-hf
- Simon Willison on llama.cpp — https://simonwillison.net/tags/llama-cpp/